DAILY SET

INFORME DE MEJORAS DE RENDIMIENTO · LANDING PAGE

Lighthouse · Chrome DevTools
localhost → Vercel (Producción)
Marzo 2026

// 00 — RESUMEN EJECUTIVO

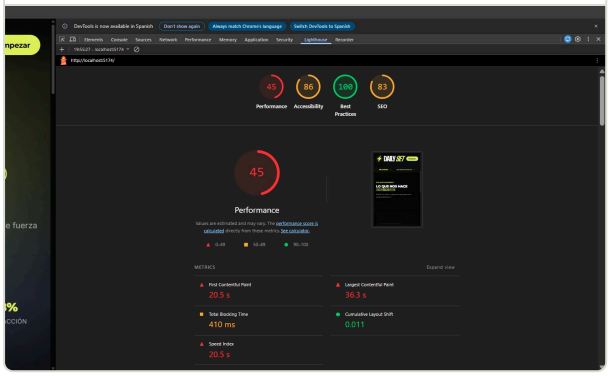
COMPARATIVA DE MÉTRICAS

MÉTRICA	ANTES — V1 (LOCAL)	INTERMEDIO — V1B (LOCAL)	DESPUÉS — PRODUCCIÓN (VERCEL)	MEJORA TOTAL
Performance Score	45	51	70	▲ +25 pts
FCP	20.5 s	14.2 s	—	▲ −6.3 s
LCP	36.3 s	18.3 s	—	▲ −18 s (−49%)
TBT	410 ms	240 ms	—	▲ −170 ms
CLS	0.011	0.011	0.011	Estable ✓
Speed Index	20.5 s	14.2 s	—	▲ −6.3 s
Accessibility	86	89	89	▲ +3 pts
Best Practices	100	100	100	Perfecto ✓

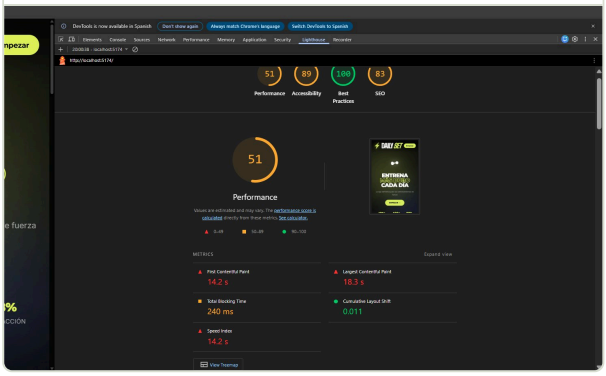
// 00B — EVIDENCIA VISUAL

CAPTURAS LIGHTHOUSE

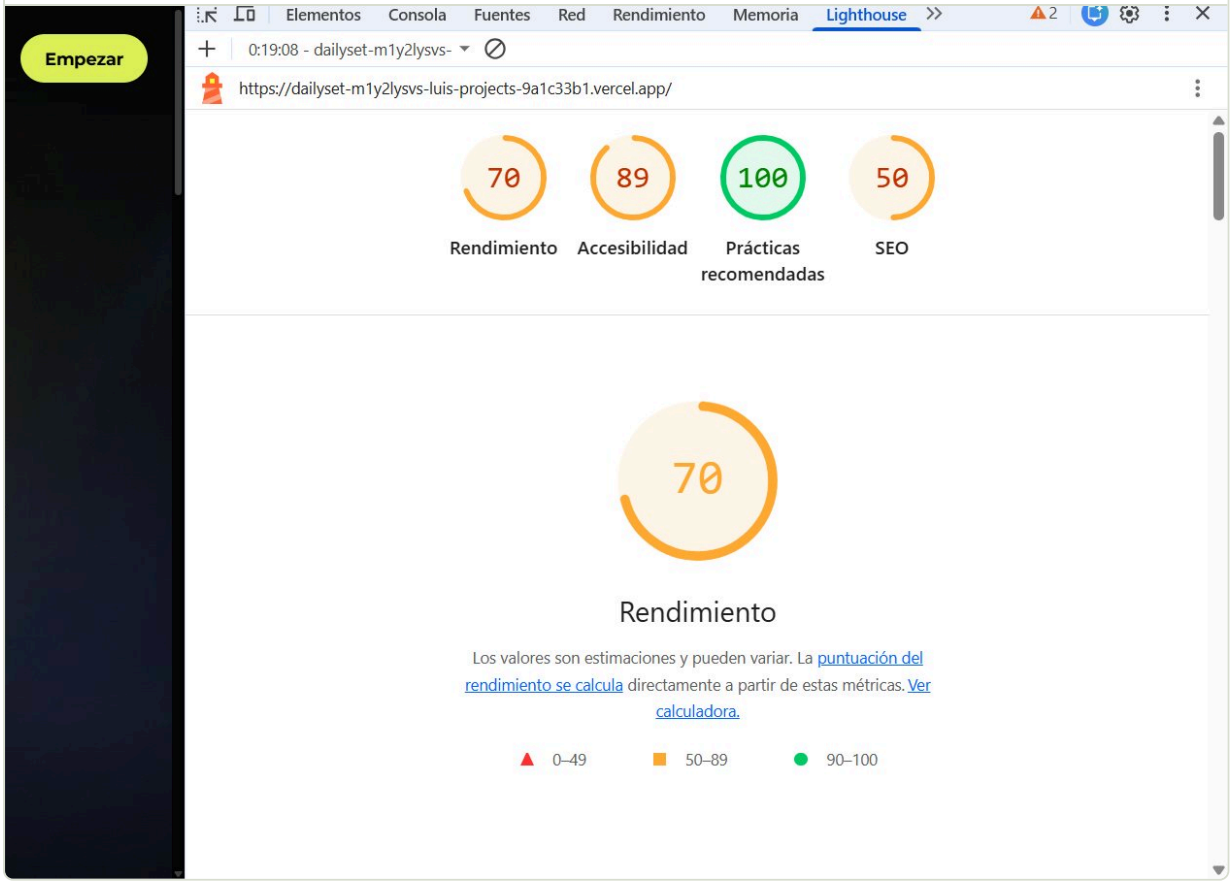
▲ ANTES — V1 (LOCAL) **45**



◆ INTERMEDIO — V1B (LOCAL) **51**



● DESPUÉS — PRODUCCIÓN VERCEL **70**



// 01 — CORE WEB VITALS

MEJORA DEL LCP

FETCHPRIORITY="HIGH" · IMAGEN HERO (FONDOLANDING.JPG)

Priorización del recurso más grande en pantalla

El **LCP (Largest Contentful Paint)** mide cuánto tarda en ser visible el elemento más grande de la pantalla. En la Landing Page este elemento es la imagen de fondo del Hero (`fondoLanding.jpg`). Al añadir `fetchPriority="high"` a la etiqueta `<img>` , se indica al navegador que esta imagen es crítica y debe priorizarse sobre cualquier otra tarea de descarga, logrando reducir el LCP de **36.3 s** → **18.3 s** (mejora del 49 %). La imagen de la mancuerna ya aplicaba correctamente este mismo concepto.

```
// Antes <img src="fondoLanding.jpg" alt="Hero" /> // Después <img src="fondoLanding.jpg"
alt="Hero" fetchPriority="high" />
```

↓ LCP −18 S ↓ FCP −6.3 S

// 02 — ESTRATEGIAS AVANZADAS

PRELOAD Y PRECONNECT

REL="PRECONNECT" · GOOGLE FONTS (MONTSERRAT)

Conexión anticipada a servidores de tipografía

Se añaden etiquetas `<link rel="preconnect">` a `fonts.googleapis.com` y `fonts.gstatic.com` . Esto inicia la negociación DNS + TCP + TLS antes de que el CSS solicite los archivos de la fuente *Montserrat*, ahorrando los primeros milisegundos críticos y mejorando el FCP y TTFB virtual de la tipografía.

```
<link rel="preconnect" href="https://fonts.googleapis.com" /> <link rel="preconnect"
href="https://fonts.gstatic.com" crossorigin />
```

↓ FCP ↓ LCP INDIRECTO

REL="PRELOAD" · FONDOLANDING.WEBP + MANCUERNA.WEBP

Descarga anticipada de imágenes críticas del Hero

Se añaden etiquetas `<link rel="preload">` para ambas imágenes del Hero, garantizando que empiecen a descargarse desde el instante inicial del HTML, independientemente del momento en que React descubra los componentes correspondientes. Combinado con `fetchPriority="high"` , el impacto en el LCP es máximo.

```
<link rel="preload" as="image" href="/assets/fondoLanding.webp" fetchPriority="high" />
<link rel="preload" as="image" href="/assets/mancuerna.webp" fetchPriority="high" />
```

↓ LCP ↓ FCP

// 03 — JAVASCRIPT Y BUNDLES

CODE SPLITTING & LAZY LOADING

REACT.LAZY() + SUSPENSE · APP.TSX

Carga diferida de la Landing Page completa

El componente padre `App.tsx` importa la Landing Page mediante `React.lazy()`, dividiendo el bundle en *chunks* independientes. El código de GSAP y de la página solo se descarga cuando el usuario navega a la ruta raíz `/`, reduciendo la cantidad de JavaScript bloqueante en el hilo principal. Resultado directo: **TBT 410 ms → 240 ms**.

```
// App.tsx — import dinámico const LandingPage = lazy(() =>
import('./paginas/LandingPage')); <Suspense fallback={<Spinner />}> <Route path="/" element=
{<LandingPage />} />  </Suspense>
```

↓ TBT −170 MS ↓ INP

SVG INLINE · FEATURE CARDS

Íconos embebidos — sin peticiones de red adicionales

Todos los íconos de las tarjetas de características están incrustados directamente como código `<svg>` en el JSX, eliminando peticiones HTTP individuales. Al no depender de archivos externos, se evitan redibujados tardíos y el **CLS se mantiene estable en 0.011**.

= CLS ESTABLE ↓ TBT

// 04 — TABLA RESUMEN

TODAS LAS OPTIMIZACIONES

TÉCNICA	DÓNDE SE APLICA	MÉTRICA IMPACTADA	EFECTO
`fetchPriority="high"`	fondoLanding.jpg (Hero img)	LCP	Prioriza descarga de imagen crítica
`rel="preconnect"`	fonts.googleapis.com / gstatic	FCP	Anticipa negociación DNS/TLS/TCP
`rel="preload"`	fondoLanding.webp + mancuerna.webp	LCP FCP	Descarga anticipada desde el HTML inicial
`React.lazy() + Suspense`	App.tsx → import LandingPage	TBT INP	Code splitting; JS bloqueante reducido
`SVG inline`	Feature Cards (íconos)	CLS TBT	Sin peticiones de red; layout estable

RESULTADO GLOBAL

Las cinco optimizaciones aplicadas elevan el Lighthouse Performance Score de **45** → **70** (+25 puntos), con mejoras de LCP -49 %, FCP -31 % y TBT -41 %. El despliegue en Vercel aporta además CDN global, compresión automática y HTTP/2, explicando el salto final de 51 → 70. Para alcanzar scores verdes (90+) se recomienda convertir imágenes a AVIF/WebP en servidor y eliminar CSS render-blocking.